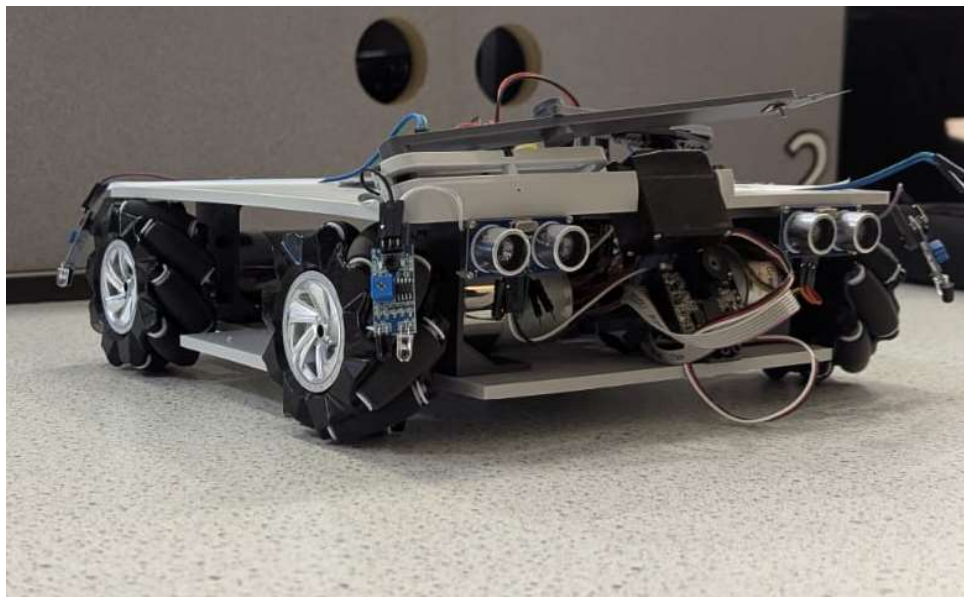


MCEN 4115/5115
Mechatronics
Robot Project Final Report
Team Eggcelsior



"LONG LIVE STEVEN, SON OF STEVE" - THE MAKER

TABLE OF CONTENTS

ABSTRACT

The goal of this project was to create a robot that could compete alongside other teams in a Mario Kart Balloon Battle-style game. Robots must be subject to specific regulations to keep matches fair. This included: robots must be fully autonomous, robots must be battery powered, team expenditure must be below \$200, robots must have a “crown” of 3 balloons equidistant from each other, and the robot must fit into a 14” x 12” footprint.

A successful robot would be able to navigate the field and use a lance and/or a shooter to pop 4 balloons on either the opposing robot or the team’s bases. To accomplish this our robot was designed with a lance as the main weapon and a disk shooter as a strong distance weapon. The robot also has many sensors for navigation including an IMU, 2 ultrasonic sensors, 4 line following sensors, and a PixyCam for balloon recognition. The chosen method for movement was omnidirectional mecanum wheels that enabled the robot to move diagonally and strafe.

CHASSIS

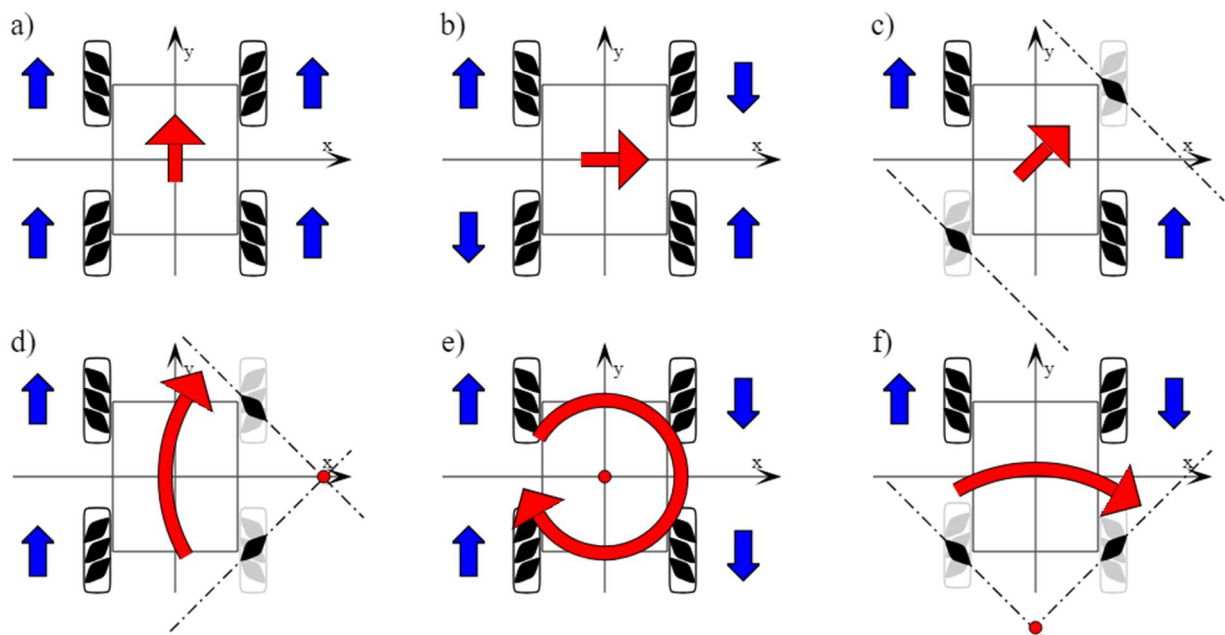
The chassis is responsible for holding all subcomponents of the robot, so the design is heavily mount-conscious. Ease of assembly and 3D printing were at the forefront of the design considerations so the body was split into a top and bottom. The bottom is where most of the electronics and motors were housed. This was a flat plate with mounting points for motor brackets the Arduino and Raspberry Pi.

The top plate of the robot was used to house more easily accessible components like the battery and the IMU which needed to be farther away from the motors to not pick up any interference. The top plate also featured passthrough holes for the motor mounts so that body clips could be used to secure the top plate to the rest of the body, but also allow for easy removal and access to the lower level.

MOVEMENT

For the robot’s movement, it was decided to use a classic 4-wheel design for simplicity. From here, mecanum wheels were chosen over regular wheels to

allow for advanced movement. This includes moving diagonally and strafing sideways.



https://en.wikipedia.org/wiki/Mecanum_wheel

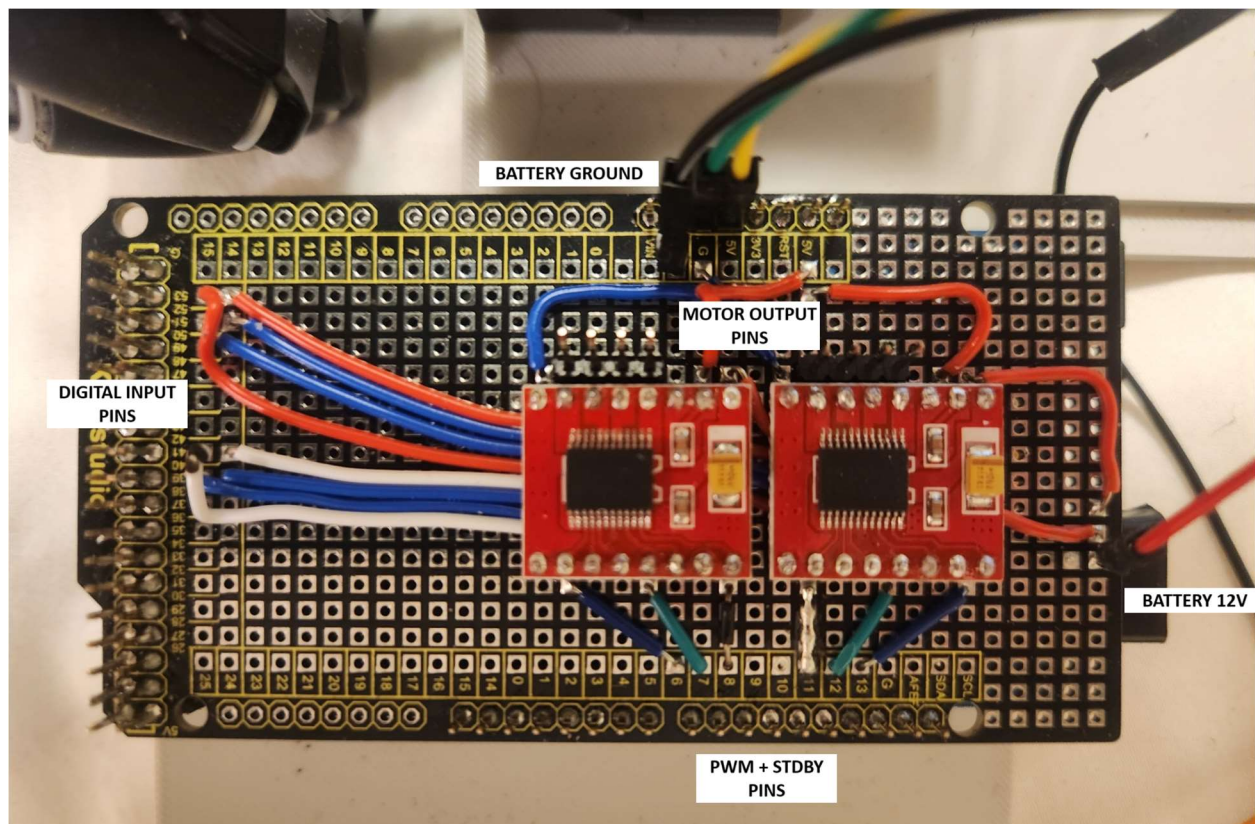
The above figure shows the movement patterns of the mecanum wheels, illustrating their ability to translate in any direction through combinations of independent wheel rotation. Being able to rotate wheels independently was crucial as it meant we could rotate about the center of the robot rather than limiting ourselves to turning over a wide radius. Though this could have been achieved with standard wheels using traditional tank steering, having the flexibility to simultaneously translate sideways allowed us to track a target by rotating while moving about in any direction.

DRIVE SYSTEM AND MOTOR CONTROL

Having originally been provided with a set of four 60RPM DC gear motors, we opted to upgrade to 300 RPM motors as it simplified the implementation of our drive system at the speed we desired. Put simply, in order to take full advantage of the maximum motor speed when using mecanum wheels to move in one direction, you must sacrifice your ability to move in another direction simultaneously. For example, if you are moving forward at full speed, you can not simultaneously rotate at that speed because all 4 wheels are already spinning at their highest RPM.

The simplest way to manage this was to design the drive system in such a way in which the speed of each mode of motion (forwards, sideways and rotational) was bound by a third of the total motor speed. The drive system incorporated one function for each mode of motion, which independently added or subtracted to the speed of each wheel depending on the motion. Simply put, the speed of the four wheels was controlled by a linear combination of the three motion patterns that govern each mode of motion (a,b, and e in the above figure), in such a way that the maximum motor speed would only be achieved if all three modes of motion were maximized simultaneously.

Physically, the drive system was controlled by an Arduino, and its overall footprint was greatly reduced through the use of a protoboard mounted above the Arduino, housing two motor drivers, and connections to each motor pin and to the battery.



LANCE

The lance was decided to be our main weapon to be used against balloons because it is more reliable, consistent, and easier to integrate than the shooter. The design used a simple scissor mechanism that relied on a single servo to extend the lance along a rail. The idea behind this design was to strike a balance between speed and control of extension, complexity, footprint, and durability. This mechanism proved to be very easy to design and implement into the design of the robot, due to its small form factor. An Xacto blade was used at the end of the lance, which provided a reliable way to pop a balloon with minimal contact.

DISK SHOOTER

The shooting mechanism was designed around the idea of using limonene to pop balloons. Limonene is a nonpolar hydrocarbon extracted from the peels of oranges that is capable of penetrating the thin latex of balloons. In order to incorporate this idea into the design, disks were developed to be extremely lightweight and feature a large groove on the outside that a bicycle front fork o-ring could slip onto. This o-ring is designed to be oil absorbent and held the limonene liquid inside, but was exposed enough so that if the disk impacted a balloon, it would release the popping agent.

The firing mechanism used a motor salvaged from a custom Nerf gun, that at maximum speed could reach 74,000 rpm. This was mounted on an adjustable motor mount and used a tapered flywheel with rubber o-ring, to grip onto disks and send them down a modified rail.

The last piece of the disk shooter was the loading mechanism. A solenoid was mounted to the back of the shooter that would actuate an arm forcing a pusher to send a disk into the flywheel. The pusher would restrict another disk from dropping down until it fully retracted, upon which when it did fully retract a new disk would be gravity-fed into the chamber ready for firing.

To power and run the solenoid and motor, 2 relay circuits were made to take in a 5V logic from an Arduino and direct the flow of the 12V lipo battery power to or away from the flywheel and solenoid.

SENSORS AND OBSTACLE AVOIDANCE

Our sensor subsystem was relatively simple. It incorporated 4 infrared sensors which were placed on each of the four corners of the chassis. This allowed us to precisely determine when the robot was approaching a wall or arena bound line, which was marked with reflective white tape. Based on which sensors were triggered, It could be determined which side of the robot was approaching an obstacle and adjust accordingly.

Additionally, two distance sensors were used, each of which was placed at the front of the robot. This choice was made on the assumption that the robot would be primarily moving forward and that the sensors would provide additional data which could be used both as a supplement and contingency for our other sensors. For example, one use case was avoiding approaching the edge of the wall in such a way that the IR sensors would straddle the marking tape on either side. The distance sensors would provide us with an alternative way to detect this obstacle. They would also allow us to detect the proximity of other robots. Having two allowed us to compare their readings to determine which side of the robot was closest to an obstacle.

To ensure efficient transmission of balloon location, IR sensor states, and distance sensor readings, the Arduino Uno firmware uses a three-byte encoding scheme. This structure minimizes communication overhead while maintaining clarity and precision in the transmitted data. Below is a detailed breakdown of the bit allocation within each byte:

Data Encoding Design

To efficiently represent the balloon's state, IR sensor activations, and distance sensor readings, the Arduino Uno firmware encodes the data into three bytes. The structure is designed to maximize information density while maintaining clarity. Below is a detailed breakdown:

Byte Structure

Byte 1: Balloon State and IR Sensor States

Bits 7-4: Balloon state information:

Bit 7: Balloon detected (0 = detected, 1 = not detected).

Bit 6: Balloon in range (1 = in range, 0 = out of range).

Bit 5: Balloon offset direction (1 = left, 0 = right).

Bit 4: Balloon offset state (1 = centered, 0 = offset).

Bits 3-0: IR sensor states:

Each bit corresponds to a binary state (1 = active, 0 = inactive) of up to 4 IR sensors.

Byte 2: Distance Sensor 1

Bits 7-0: Scaled value of the first distance sensor (8-bit resolution).

Byte 3: Distance Sensor 2

Bits 7-0: Scaled value of the second distance sensor (8-bit resolution).

Encoding Logic

1. Balloon State Packing: Balloon status is represented by assigning specific bits for detection, range, offset direction, and offset state. For example:

```
balloon_status = (detected << 7) | (in_range << 6) | (offset_dir << 5) |  
(offset_state << 4);
```

2. IR Sensor Packing: IR sensor states are combined into the lower 4 bits of Byte 1:

```
tap_status = (tap1 << 3) | (tap2 << 2) | (tap3 << 1) | tap4;  
Byte1 = balloon_status | tap_status;
```

3. Distance Sensors: Sensor readings are scaled and directly stored in Bytes 2 and 3.

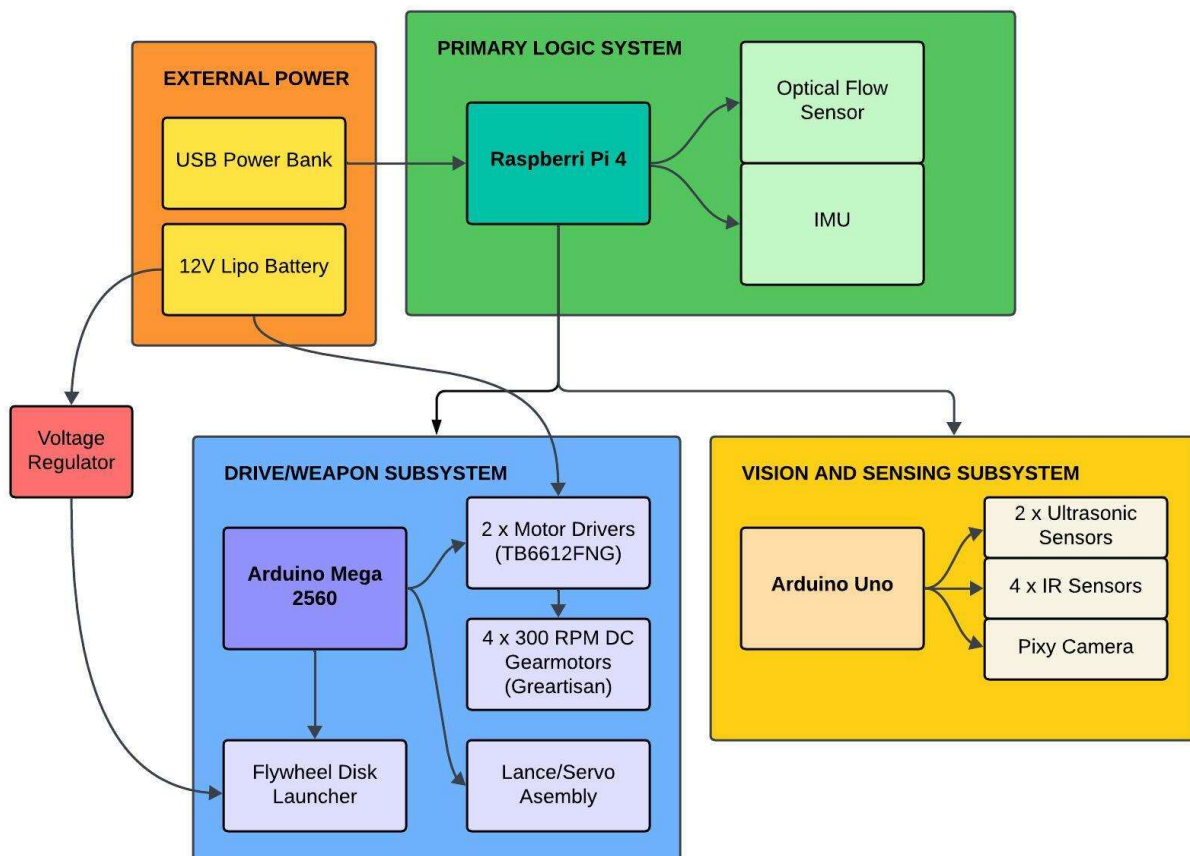
COMPUTER VISION

Our team's sensor system also incorporated a Pixy Camera which was used to detect balloons of a specified color. The integration of the camera revolved around tracking the location of the balloon on the sensor and adjusting the movement of the robot based on that location relative to the

centerline. In addition, by placing the camera much lower to the ground than the 6-inch high plane prescribed for the balloons, and at an angle facing upwards, our team could use the camera to detect whether or not a balloon was in range of our weapon system, without having to rely on the size of the block detected by the camera, which was deemed to be somewhat unreliable.

COMMUNICATION

INTEGRATION



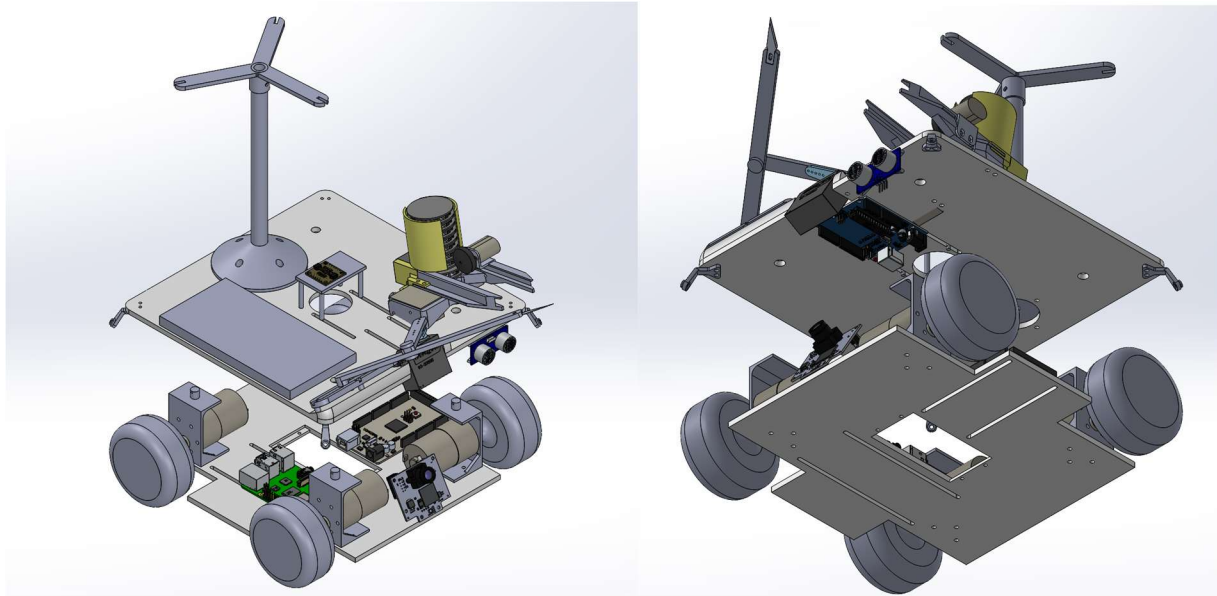
The Raspberry Pi handled the primary logic system, functioning as the “brains” of the robot. Based on sensor inputs from the Arduino Uno, front and rear collision states were assigned, and distances to objects on the left and right determined a "pan" value. It updates a data_array with motion

commands that are sent to the Arduino Mega which handles the drive system and weapon system.

Key functionalities include:

1. Collision Handling: Determines actions based on whether there are collisions at the front, rear, both, or none.
2. Object Distance Evaluation: Adjusts movement directions based on object distances (dist_L and dist_R) to avoid collisions or navigate around obstacles.
3. Pan Sensor Analysis: Uses the "pan" value to identify object position (left, right, or invalid) and adjust data_array.
4. Timer-Based State Control: Changes actions dynamically within specified time intervals after a collision is detected.
5. Weapon State Handling: Determines the state of the lance and shooting mechanism based on the data received by the Arduino Uno.
6. Data Transmission: Sends updated motion and weapon commands (data_array) to a microcontroller (ser_mega).

FINAL DESIGN



BILL OF MATERIALS (B.O.M)

Part	Part Number	Qty.	Price	Source
Primary Logic System				
Raspberry Pi 4	-	1	-	Provided
IMU	-	1	\$20	Amazon
Flow Sensor	-	1	\$8	Amazon
Drive/Weapon Subsystem				
Arduino Mega 2560	-	1	-	Provided
Motor Drivers	TB6612FNG	2	\$8	Amazon
Greartisan 300 RPM Gearmotors	-	4	\$60	Amazon
Mecanum Wheels	-	4	\$40	
74,000RPM Flywheel Motor	-	1	-	Salvaged
Vision and Sensing Subsystem				
Arduino Uno	-	1	-	Provided

Ultrasonic Sensors	-	2	-	Provided
IR Sensors	-	4	-	Provided
Pixy Camera	-	1	-	Provided
External Power				
USB Power Bank	-	1	-	Provided
12V Zeee Lipo Battery	-	2	-	Provided
Miscellaneous				
Voltage Regulator	-	1	\$6	Amazon
Chassis Parts	-	N/A	~\$18	3D Printed
Hardware (screws, pins, etc)	-	N/A	~\$10	Amazon
TOTAL	\$170			

CONCLUSION